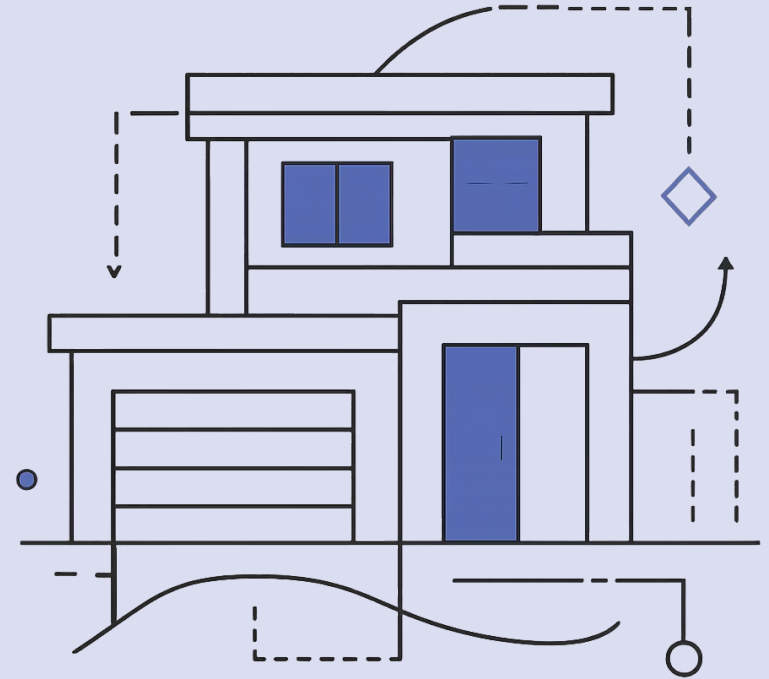


Multiple Linear Regression

Predicting a number using many input features — illustrated through house price prediction.

BEGINNER ML

REGRESSION



Beyond Simple Linear Regression

In Simple Linear Regression, we used just one input: Area $\rightarrow$ Price. But real house prices depend on many factors.

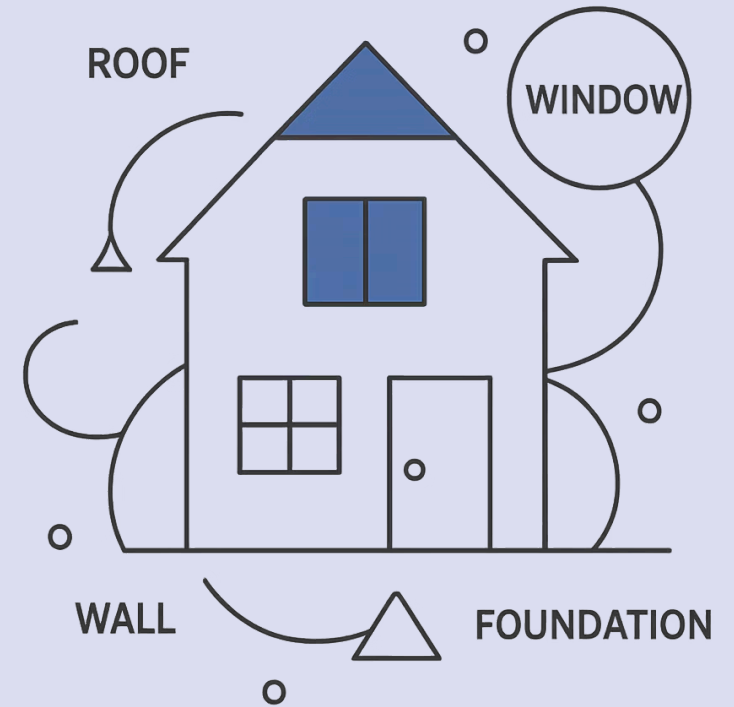
Bedrooms

House Age

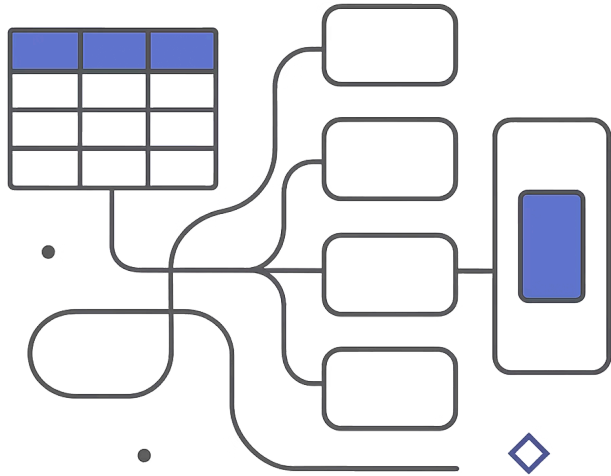
Distance from City

Location & Metro

We need a model that uses many input features — that model is [Multiple Linear Regression](#).



What is Multiple Linear Regression?



Definition

Predicting one numerical output using more than one input feature.

House Price Example

Area + Bedrooms + House Age + Distance → Price

- Inputs = multiple features
- Output = one numerical target

Why "Multiple"?

Because we use **multiple input features** to predict one target.

Area

Size of the house

Bedrooms

Number of bedrooms

House Age

How old the house is

Distance

Distance from city center

Multiple inputs → One output (Price)

Why "Linear"?

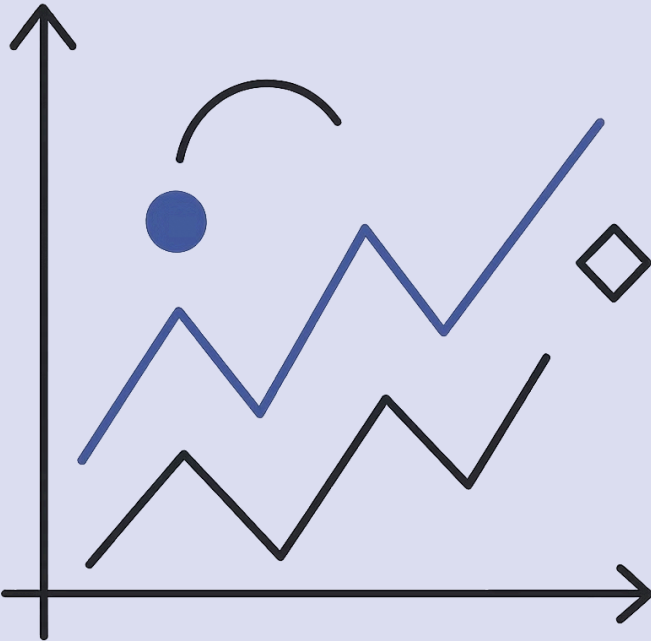
The model assumes a straight-line type relationship between each feature and the target.

→ Area ↑
Price increases

→ Bedrooms ↑
Price increases

→ House Age ↑
Price decreases

→ Distance ↑
Price decreases



Simple vs. Multiple: The Formula

Simple Linear Regression

$\text{Price} = \text{slope} \times \text{Area} + \text{intercept}$

Multiple Linear Regression

$\text{Price} = b_0 + b_1 \times \text{Area} + b_2 \times \text{Bedrooms} + b_3 \times \text{Age} + b_4 \times \text{Distance}$

 Each feature gets its own coefficient: b_0 = intercept, b_1 – b_4 = feature weights.

What is a Coefficient?

A coefficient is the **importance or effect value** the model learns for each feature.

Price = 10 + 0.04×Area + 5×Bedrooms - 1×Age - 2×Distance

Feature	Coefficient
Area	0.04
Bedrooms	5
Age	-1
Distance	-2

Positive vs. Negative Coefficients

Positive Coefficient

Feature increases $\rightarrow$ Target increases

Area $\uparrow \rightarrow$ Price $\uparrow$

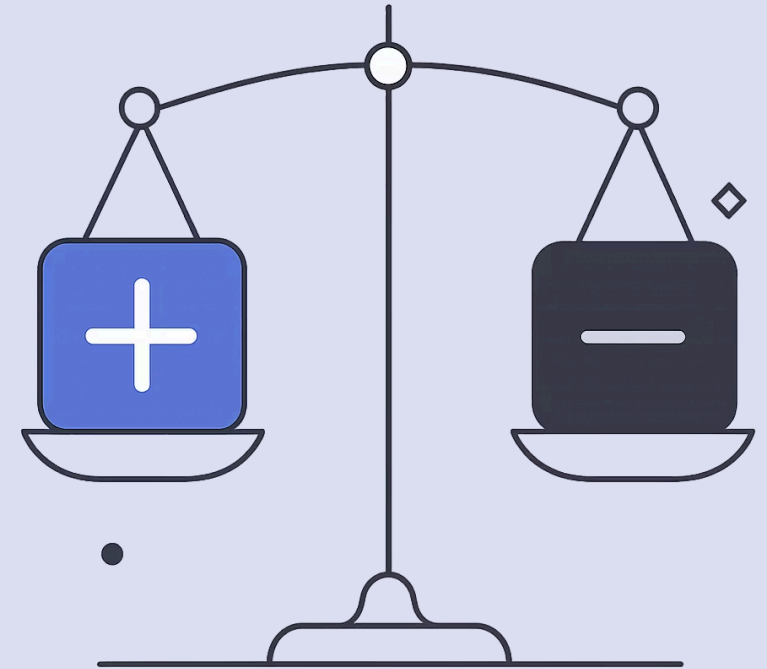
Bedrooms $\uparrow \rightarrow$ Price $\uparrow$

Negative Coefficient

Feature increases $\rightarrow$ Target decreases

Age $\uparrow \rightarrow$ Price $\downarrow$

Distance $\uparrow \rightarrow$ Price $\downarrow$



Understanding Each Coefficient

$$\text{Price} = 10 + 0.04 \times \text{Area} + 5 \times \text{Bedrooms} - 1 \times \text{Age} - 2 \times \text{Distance}$$

Area (0.04)

+1 sqft → price up ₹4,000. +100 sqft → price up ~₹4 lakh.

Bedrooms (5)

Each extra bedroom → price up ~₹5 lakh.

Age (-1)

Each year older → price down ~₹1 lakh.

Distance (-2)

Each km farther → price down ~₹2 lakh.

Manual Calculation Example

New House

Feature	Value
Area	1500 sqft
Bedrooms	3
Age	4 years
Distance	6 km

Calculation

$$\begin{aligned} \text{Price} &= 10 + 0.04 \times 1500 \\ &\quad + 5 \times 3 - 1 \times 4 - 2 \times 6 \\ &= 10 + 60 + 15 - 4 - 12 \\ &= 69 \end{aligned}$$

✔ Predicted Price: ₹69 lakh

Why Multiple is Better

Two houses with the same area can have very different prices:

Area	Bedrooms	Age	Distance	Price
1500	3	3	5 km	₹82 lakh
1500	2	12	15 km	₹55 lakh

 Simple Linear Regression can't explain this difference. Multiple features are needed.

Dataset Example

Training data for house price prediction:

Area	Bedrooms	Age	Distance	Price (₹L)
1000	2	5	8	55
1500	3	3	5	82
2000	4	2	4	120
2500	4	1	2	155

Features: Area, Bedrooms, Age, Distance | Target: Price

PYTHON STEP 1

Import Libraries

```
import pandas as pd  
from sklearn.linear_model import LinearRegression
```

pandas

Create and handle tabular data

LinearRegression

Train the regression model

PYTHON STEP 2

Create the Dataset

```
data = {  
    "area": [1000,1500,2000,900,1800,2200,1300,2500],  
    "bedrooms": [2, 3, 4, 2, 3, 4, 2, 4],  
    "age": [5, 3, 2, 10, 4, 2, 6, 1],  
    "distance": [8, 5, 4, 12, 6, 3, 9, 2],  
    "price": [55, 82, 120, 42, 95, 130, 65, 155]  
}  
df = pd.DataFrame(data)  
print(df)
```

This creates our sample house price dataset with 8 rows.

PYTHON STEP 3

Separate Features and Target

```
X = df[["area", "bedrooms", "age", "distance"]]  
y = df["price"]
```

X — Features (Input)

area, bedrooms, age, distance

Multiple input columns

y — Target (Output)

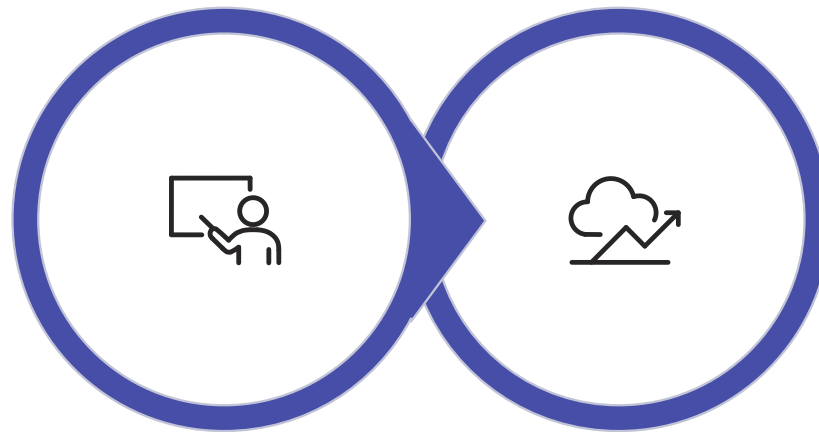
price

One output column

Train the Model

```
model = LinearRegression()  
model.fit(X, y)
```

- ❗ `model.fit(X, y)` learns how area, bedrooms, age, and distance affect price — finding the best coefficient for each feature.



Fit

Predict

PYTHON STEP 5

Predict a New House Price

```
new_house = pd.DataFrame({  
    "area": [1700], "bedrooms": [3],  
    "age": [4], "distance": [6]  
})  
predicted_price = model.predict(new_house)  
print("Predicted price:", predicted_price[0], "lakhs")
```

- ✅ The model outputs one numerical value: the predicted house price in lakhs.

Full Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression

data = {
    "area": [1000,1500,2000,900,1800,2200,1300,2500],
    "bedrooms": [2,3,4,2,3,4,2,4],
    "age": [5,3,2,10,4,2,6,1],
    "distance": [8,5,4,12,6,3,9,2],
    "price": [55,82,120,42,95,130,65,155]
}
df = pd.DataFrame(data)
X = df[["area","bedrooms","age","distance"]]
y = df["price"]

model = LinearRegression()
model.fit(X, y)

new_house = pd.DataFrame({
    "area":[1700],"bedrooms":[3],
    "age":[4],"distance":[6]
})
print("Predicted price:", model.predict(new_house)[0], "lakhs")
```

Checking What the Model Learned

After training, inspect the learned coefficients:

```
print("Intercept:", model.intercept_)  
print("Coefficients:", model.coef_)  
  
coefficients = pd.DataFrame({  
    "Feature": X.columns,  
    "Coefficient": model.coef_  
})  
print(coefficients)
```

 This shows the intercept (base value) and the coefficient for each feature.

Reading the Coefficients

Feature	Coefficient	Meaning
area	positive	Larger area → higher price
bedrooms	positive	More bedrooms → higher price
age	negative	Older house → lower price
distance	negative	Farther from city → lower price

📌 For beginners: focus on the sign (direction), not the exact value.



Warning About Coefficients

Do not blindly compare coefficient sizes — features are on different scales.

Area

1000 – 3000

Bedrooms

1 – 5

Age

1 – 30

Distance

1 – 50



Beginner-safe rule: Coefficient sign tells direction. Positive = increases target. Negative = decreases target.



Train-Test Split

In real projects, never train and test on the same data. Split first:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

 Train on training data. Predict on testing data. Compare actual vs. predicted prices.

Common Beginner Mistake

❌ Wrong

```
X = df[["area", "bedrooms",  
        "age", "distance",  
        "price"]]  
y = df["price"]
```

Price is inside the features — the model already sees the answer!

✅ Correct

```
X = df[["area", "bedrooms",  
        "age", "distance"]]  
y = df["price"]
```

Golden rule: Target column must never be inside the features.

Another Example: Salary Prediction

Multiple Linear Regression applies beyond housing:



Experience

Years in the field



Skill Score

Technical ability rating



Interview Score

Performance in interview



Education Level

Degree attained



$$\text{Salary} = b_0 + b_1 \times \text{Experience} + b_2 \times \text{SkillScore} + b_3 \times \text{InterviewScore} + b_4 \times \text{EducationLevel}$$

Simple vs. Multiple: Side by Side

Point	Simple LR	Multiple LR
Input features	One	More than one
Example	Area $\rightarrow$ Price	Area + Bedrooms + Age $\rightarrow$ Price
Formula	$y = mx + c$	$y = b_0 + b_1x_1 + b_2x_2 + \dots$
Real-world use	Basic	More practical
Visualization	Easy (2D line)	Harder (multi-dim)

When to Use Multiple Linear Regression

- ☐ Numerical target
Output is a number (price, salary, sales)
- ☐ Multiple features
More than one input affects the target
- ☐ Roughly linear
Relationships are approximately straight-line
- ☐ Explainability needed
You want to understand feature effects



Use Cases at a Glance



House Price

Area, bedrooms, age,
distance



Salary

Experience, skills,
education



Sales Forecast

Season, marketing spend,
region



Electricity Use

Temperature, appliances,
occupancy

Limitations

Multiple Linear Regression may not work well when:

Non-linear relationships

Patterns are curved, not straight-line

Complex feature interactions

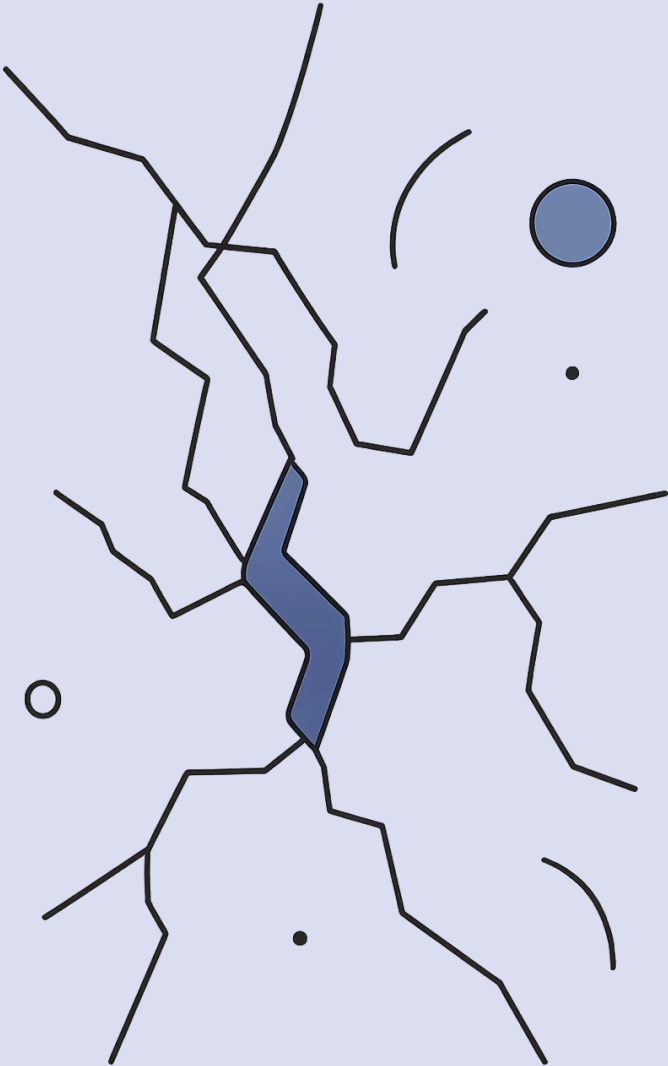
Features combine in non-additive ways

Outliers or poor data quality

Skews the learned coefficients

Missing key features

E.g., location quality missing → weak predictions



Final Summary

Multiple Linear Regression = many input features + one numerical target

Formula

$$\text{Price} = b_0 + b_1 \times \text{Area} + b_2 \times \text{Bedrooms} + b_3 \times \text{Age} + b_4 \times \text{Distance}$$

Simple LR

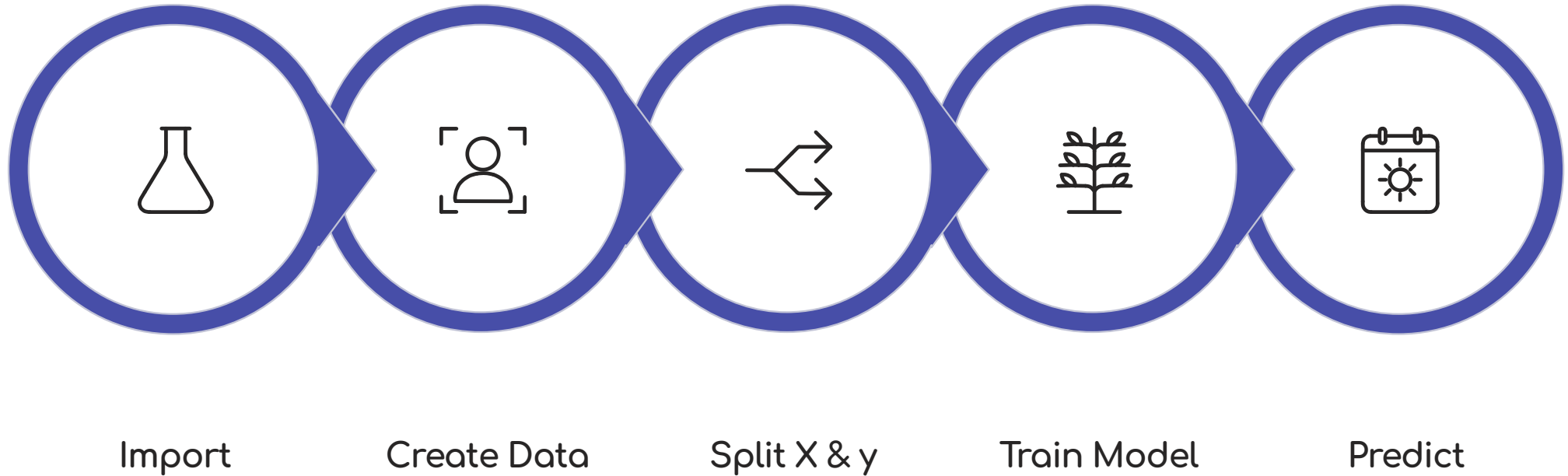
One input → one output

Multiple LR

Many inputs → one output

Both predict a number. Multiple LR is simply more realistic.

The Full Pipeline



This is the standard workflow for any Multiple Linear Regression project in Python using scikit-learn.

Key Concepts Recap

01

Multiple Features

Use more than one input column to predict the target

02

Coefficients

Each feature gets a weight; sign shows direction of effect

03

Formula

$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$

04

Train-Test Split

Always separate training and testing data in real projects

05

Target Separation

Never include the target column inside the features



Check Your Understanding

A company wants to predict salary using: Experience, Skill Score, Interview Score, and Education Level. What type of ML problem is this?

1

Supervised Learning

We have labeled training data

2

Regression

Target (salary) is a number

3

Multiple Linear Regression

Multiple input features



You've Got It!

Multiple Linear Regression is one of the most practical tools in machine learning — simple, explainable, and powerful.

House Price

Area + Bedrooms + Age + Distance → Price

Salary

Experience + Skills + Education → Salary

Next up: Polynomial Regression & Feature Scaling